Problem Statement:Write a program to solve a 0-1 Knapsack problem using dynamic programming or branch and bound strategy.

Code:

```
import java.util.Scanner;

public class Knapsack {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Input number of items
        System.out.print("Enter number of items: ");
        int n = sc.nextInt();

        int[] value = new int[n];
        int[] weight = new int[n];

        // Input values and weights
        for (int i = 0; i < n; i++) {
            System.out.print("Enter value of item " + (i + 1) + ": ");
            value[i] = sc.nextInt();

            System.out.print("Enter weight of item " + (i + 1) + ": ");
            weight[i] = sc.nextInt();
        }

        // Input knapsack capacity
        System.out.print("Enter knapsack capacity: ");
        int capacity = sc.nextInt();

        // DP table
        int[][] dp = new int[n + 1][capacity + 1];

        // Dynamic Programming
        for (int i = 1; i <= n; i++) {

            for (int w = 1; w <= capacity; w++) {

                if (weight[i - 1] <= w) {

                    dp[i][w] = Math.max(
                            value[i - 1] + dp[i - 1][w - weight[i - 1]],
                            dp[i - 1][w]
                    );

                } else {
```

```
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    // Display maximum value
    System.out.println("\nMaximum value = " + dp[n][capacity]);

    // Find selected items
    int w = capacity;

    System.out.println("Selected items:");

    for (int i = n; i > 0; i--) {

        if (dp[i][w] != dp[i - 1][w]) {

            System.out.println(
                    "Item " + i +
                    " (Value = " + value[i - 1] +
                    ", Weight = " + weight[i - 1] + ")"
            );

            w = w - weight[i - 1];
        }
    }

    sc.close();
  }
}
```

Output:

```
comp@comp-HP-ProDesk-600-G3-SFF:~/Aniket$ javac Knapsack.java
comp@comp-HP-ProDesk-600-G3-SFF:~/Aniket$ java Knapsack
Enter number of items: 4
Enter value of item 1: 120
Enter weight of item 1: 20
Enter value of item 2: 150
Enter weight of item 2: 40
Enter value of item 3: 180
Enter weight of item 3: 60
Enter value of item 4: 200
Enter weight of item 4: 80
Enter knapsack capacity: 60
Maximum value = 270
Selected items:
Item 2 (Value = 150, Weight = 40)
Item 1 (Value = 120, Weight = 20)
```